

Proyecto

Implementación de un juego basado en la idea de [Roguelike](#) ASCII

Objetivo

El proyecto consiste en el desarrollo de un juego de rol (tipo roguelike) basado en texto que soporte las acciones más básicas de este tipo de juegos, como mover a un jugador a través de una mazmorra, recoger / equipar objetos o pelear con enemigos. El objetivo básico del juego consistirá en llevar al jugador desde la posición inicial de la mazmorra hasta una posición final sano y salvo. Cuanta más vida y energía tenga el jugador y menos pasos haya necesitado para llegar al final, tanto mejor será su puntuación.

En este tipo de juegos, el usuario dispone de una serie de acciones que puede realizar y que introduce en modo texto en un terminal. Estas acciones por lo general son del tipo “coger objeto” para recoger un objeto que se encuentra en el suelo; “mirar” para obtener una descripción del lugar actual y/o alrededores; “usar objeto” para obtener algún beneficio o realizar cierta acción; “dejar objeto” para soltarlo en la posición actual, “norte” para desplazarse una casilla hacia el norte, etc. Cada vez que se ejecuta una acción sucede un cambio en el estado del juego, como por ejemplo cambia la casilla en la que se encuentra el jugador si éste se ha movido o aumenta su vida si ha decidido usar un objeto especial como una poción mágica. Cada vez que el usuario realiza una acción se imprime por consola la información de lo que ha sucedido y vuelve de nuevo a esperar hasta que el usuario inserte de nuevo la acción siguiente que va a realizar. A continuación se muestra una secuencia para un ejemplo de juego:

Ejemplo de interacción con el juego desde el punto de vista del jugador.

Bienvenido al juego “Hora de Aventuras”. Como gran aventurero que eres, deberá recorrer el país de Ooo en busca de los planes del malvado Rey Hielo para comunicárselo cuanto antes a la princesa Chicle en el reino de Chuchelandia.

Introduce el nombre de tu personaje:

nombre> Finn

Finn entra en el mapa.

Estás en un *pasillo*. Está oscuro y hay candelabros a ambos lados del mismo.

Los posibles movimientos son norte, este, oeste.

Finn [salud(100) energía(80)] accion > mirar

No encuentras nada.

Finn [salud(100) energía(80)] accion> avanzar norte

Llegas a lo que parece una gran sala con un techo alto y decorado con figuras.

Los posibles movimientos son sur, norte, este.

Finn [salud(100) energía(70)] accion> mirar

Encuentras 'pocima energia gunter'.

Encuentras 'espada Scarlet'

Finn [salud(100) energía(70)] accion> coger "pocima energia gunter"

Recoges y guardas en tu mochila 'pocima energia gunter'.

Tu mochila se encuentra al 8% de su capacidad.

Finn [salud(100) energía(70)] accion> mochila

Tu mochila se encuentra al 8% de su capacidad.

Tu mochila contiene:

pocima energia gunter [+20 de energia]

Finn [salud(100) energía(70)] accion> usar pocima energia gunter

Recuperas 20 de energía.

Finn [salud(100) energía(90)] accion> avanzar este

Estás en una habitación muy grande. Está oscuro salvo por la luz que entra por una ventana rota.

Finn [salud(100) energía(80)] accion> ...

Existen muchos juegos de este tipo a los cuales se puede jugar con tan solo un terminal. Un ejemplo de juego en español es Balzhur. Para probarlo, basta con usar telnet para conectarse a la máquina y comenzar a jugar:

`$ telnet balzhur.genesismuds.com 5400`

Requisitos funcionales del programa

Los requisitos funcionales indican las características de funcionamiento y comportamiento que el programa o juego deberá presentar para que se considere completo. Ejemplos de requisitos funcionales serían "el jugador podrá desplazarse en 4 direcciones sobre el mapa" o "el jugador podrá recoger objetos para introducir en su mochila". A continuación se detallan todos los requisitos funcionales del proyecto:

- El programa soportará todas las operaciones necesarias para permitir el movimiento del personaje por el mapa. El mapa será rectangular (2D) con unas dimensiones arbitrarias y el movimiento del personaje estará limitado por dichas dimensiones.
 - El mapa estará dividido en casillas o celdas y deberá especificar en qué casilla comenzará el juego y cual es la casilla objetivo del juego.

- El mapa también deberá tener un nombre y una descripción que deberán mostrarse a la hora de iniciar el juego para situar al jugador en la escena de juego.
- Las celdas deberán tener una descripción, por ejemplo “pasillo estrecho” y podrán contener una lista de objetos.
- No todas las celdas serán transitables. Habrá celdas por las que el jugador se puede mover y otras que el jugador no podrá cruzar. Dependiendo del tipo de celdas vecinas el jugador podrá moverse sólo en ciertas direcciones. Por ejemplo, si la celda donde se encuentra el jugador no tiene ni al este ni al oeste una celda vecina transitable pero sí las hay al norte y al sur, el jugador tendrá sólo dos posibles movimientos, NORTE y SUR.
- El juego deberá soportar el pintado de mapas en modo ASCII, es decir, se deberá poder imprimir el mapa del juego con caracteres por filas y columnas. Esto puede ser interesante para crear y depurar mapas o por ejemplo para añadir al juego la posibilidad de ver el mapa del mundo si el jugador dispone de un mapa. Ejemplo de representación ASCII y gráfica [aquí](#).
- A lo largo del mapa deberá haber objetos distribuidos que el jugador podrá recoger y/o usar. Estos objetos tienen un peso determinado, un nombre, un tipo y un efecto. Por ejemplo, podríamos tener un objeto tipo “poción”, con un nombre descriptivo “poción mágica pequeña”, un peso de 0.2 kg y un efecto +5 al número de unidades de salud recuperadas al usarlo. En este caso, el efecto sería las unidades de vida que recuperan al jugador. Un veneno en cambio tendría un efecto negativo. Los objetos que al menos debe tener el juego son:
 - Poción de salud: recupera un número de puntos de vida.
 - Poción de energía: recupera un número de puntos de energía.
 - Veneno: resta un número de puntos de vida al jugador.
 - Mapa: sería un ejemplo de objeto sin efecto que puede ser usado. El jugador puede usarlo sólo si lo lleva en la mochila, y al ser usado se imprimirá el mapa completo del juego.
- El personaje principal siempre se encuentra en alguna celda del mapa desde el momento en que empieza el juego. Su objetivo es moverse desde la casilla inicial hasta la casilla final del mapa.
 - Este personaje tendrá una cantidad de vida que deberá mantener para poder seguir jugando. En el momento en que la vida se agote, el juego terminará.
 - El personaje tiene muy buena memoria, con lo cual sabe en todo momento por qué sitios del mapa pasó desde su entrada. Cuando termine el juego (bien o mal), deberá imprimirse por pantalla el camino seguido por el jugador.
 - El personaje dispondrá de una mochila donde podrá guardar aquellos objetos que vaya encontrando por el mapa. Además, la mochila tiene una capacidad limitada (número de objetos máximo) y un peso máximo, y no se podrán añadir objetos a la mochila si se excede la capacidad o el peso máximo de los objetos que soporta.

- Además, para que el personaje pueda moverse por el mapa, deberá disponer de una cantidad suficiente de energía que le permita moverse. Si esta energía del personaje se agota, el juego terminará. Cuanto más peso tenga que soportar en su mochila, más se cansará al moverse.
- El juego deberá tener un nombre y una descripción y contará con restricciones globales al juego. Teniendo esto en cuenta:
 - La interfaz del juego estará basada en texto y la interacción del jugador se reducirá a imprimir texto por pantalla y leer texto de teclado para obtener la acción que desea realizar el jugador en cada momento. Esto en la programación de videojuegos es conocido como “main game loop” o bucle principal. Este bucle se repite continuamente hasta salir del juego, ejecutando una serie de acciones principales, como procesar la entrada del jugador (acción a realizar), actualizar el estado del juego e imprimir la información por la pantalla (y así sucesivamente).
 - Al inicio del juego, se deberá preguntar al jugador el nombre que quiere que tenga su personaje en el juego.
 - Se indicará el número máximo de pasos que podrá dar el personaje para llegar a la casilla final del juego.
 - El juego deberá mostrar el estado del jugador en todo momento, es decir, deberá mostrar en cada iteración (ciclo) del bucle principal su nivel de salud (o vida) y su energía disponible (por ejemplo justo antes de pedir la acción al usuario).
 - El jugador podrá realizar las siguientes acciones:
 - Mirar: el jugador podrá mirar en el mapa para ver si hay algún objeto que pueda recoger.
 - Mover: el jugador podrá mover su personaje por todas las casillas del mapa en las que tenga permitido andar. En cada movimiento se podrán indicar el número de casillas por las que pasará. Habrá cuatro direcciones de movimiento correspondientes con los cuatro puntos cardinales.
 - Ojear inventario: el jugador podrá ver qué objetos tiene guardados en su inventario.
 - Coger objeto del mapa: Según el tipo objeto pueden pasar dos cosas, que le haga efecto al momento al personaje y no se coja, o que se pueda coger y pase a formar parte de la mochila (si cabe).
 - Tirar objeto en el mapa: Si el jugador quiere liberar la carga de su mochila, se le permitirá tirar los objetos guardados. Cuando se tira un objeto, pasará a estar disponible en la celda actual del mapa.
 - El programa deberá disponer de un comando “ayuda” que informará al jugador de los comandos de los que dispone el juego para interaccionar con el mismo.

Evaluación de la 1ª entrega del proyecto

En esta primera entrega **no habrá que implementar todos** los requisitos del proyecto. Los únicos que se evaluarán⁽¹⁾ serán los siguientes.

		Puntuación
1	La implementación debe constar de: • Al menos 6 clases. • Con los atributos y modificadores de acceso correctamente definidos. • Con los getters que permitan recuperar los valores de los atributos. Si no son necesarios se deberán justificar.	1.5
2	Cuatro de las clases tendrán, al menos, cuatro atributos que serán de tres tipos distintos.	0.5
3	Los setters limitan correctamente los valores de todos los atributos de las clases.	1
4	Cuatro de las clases tienen al menos dos constructores que inician correctamente los atributos de las clases a las que pertenecen.	0.5
5	Las clases pertenecen a dos paquetes diferentes.	0.5
6	En dos clases distintas aplicar sobrecarga de métodos.	1

La interacción del jugador con el juego es a través de una interfaz de comandos. Por lo tanto, el usuario introducirá en la consola un comando, el intérprete de comandos del juego trasladará la solicitud al método/clase encargada de procesar el comando y devolverá el resultado correspondiente, actualizando, si es necesario, los demás elementos gráficos que se muestran en la pantalla. Dicha interfaz textual debe:

		Puntuación
7	Representar gráficamente en la pantalla el mapa después de cada movimiento, incluyendo posición del personaje y celda objetivo. Deberán representarse todas las celdas del mapa utilizando un código ASCII a elección que permita diferenciar las diferentes celdas del mapa.	3
8	Representar el estado del jugador en todo momento, es decir, su salud y energía.	1
9	Al iniciar el juego se deberá pedir por pantalla el nombre del personaje para personalizar la partida.	1

Además, en esta primera entrega la interfaz textual deberá soportar los siguientes comandos:

		Puntuación
10	Mover al personaje en las 4 direcciones (NORTE, SUR, ESTE, OESTE) mediante	5

	comandos, siguiendo las restricciones establecidas por el mapa donde se encuentre el personaje (tiene que haber celdas que el jugador no pueda atravesar). Ej: "mover norte"	
11	Mostrar la descripción de la celda actual junto con la lista de objetos contenidos en la celda donde se encuentra el jugador. Ej: "mirar"	3
12	Listar la secuencia de movimientos del jugador desde el inicio del juego hasta el momento actual. Ej: "recorrido"	2

- (1) Si los ítems no se cumplen totalmente, se considerará que no se obtiene ningún punto. Por ejemplo, si falta un getter correspondiente a un atributo de una clase, el ítem 1 se puntúa como 0.